ENGINEERING FIELD REPORT

Fitting Qwen3.8-Flash-Next on one 96 GB Blackwell GPU

How a signed-W4 PLE sidecar, pinned-host UVA gather, and an SM120 XQA patch made a 135 GB NVFP4 checkpoint practical on 64 GB of system RAM.

47,684 -> 26,822 GiB	131,072	258.3 tok/s
host PLE table	qualified context	c4 aggregate

Tested on Dell Pro Max Tower T2 / RTX PRO 6000 Blackwell Workstation Edition / Ubuntu 24.04.4

Configuration captured 26 August 2026. Private network identifiers and credentials are intentionally omitted.

EXECUTIVE SUMMARY

The result in one page

This deployment runs the RadixArk Qwen3.8-Flash-Next NVFP4 candidate on one RTX PRO 6000 Blackwell Workstation Edition. The checkpoint is about 135.195 GB of tensors. Routed experts are NVFP4; the original FP8 PLE n-gram table remains host-resident. Stock pinned PLE offload could not allocate its 47.684 GiB table on a nominal 64 GB host. The working design stores that table as signed INT4 with E4M3 block scales, cutting the pinned allocation to 26.822 GiB.

20.862 GiB	81.29 GB	~203 s	4
pinned RAM saved	GPU weight load	cold start to ready	shared request slots

What is proven on this box

- The prebuilt sidecar loads into two pinned CPU tensors in about 16 seconds, versus about 309.5 seconds for online packing.
- A Triton kernel gathers only selected PLE rows through CUDA UVA, unpacks signed nibbles, applies E4M3 block scales, and emits BF16 rows. The full table never enters VRAM.
- The production profile serves 131,072-token contexts with a roughly 160K shared KV pool, four request slots, BF16 KV, FP32 Mamba state, and decode CUDA graphs at batch 1, 2, and 4.
- Exact-output, JSON, tool, reasoning-separation, long-retrieval, boundary, and concurrency gates passed. A broader parallel-tool test exposed prompt sensitivity and is reported later rather than hidden.

What this document is - and is not

Experimental downstream work
The W4 PLE loader, sidecar builder, and SM120 routing overlay are downstream patches, not a released SGLang feature. The RadixArk card describes the checkpoint as a private candidate even though its exact tested revision is currently public and ungated; its license remains 'other' inherited from the source model. Reproduction and redistribution still require compliance with the Qwen terms.

Section	Question answered
Hardware and pinned stack	What exact machine and software were used?
Memory fit	Why did stock fail, and what changed byte-for-byte?
Runtime integration	How are PLE rows gathered without moving the table to VRAM?
Serving profile	Which SGLang flags, kernels, caches, and graph settings are active?
Evidence	What performance, context, correctness, and failure behavior were observed?
Reproduction	What must another enthusiast build, pin, verify, and test?

HARDWARE

Exact test machine

Component	Deployed value	Why it matters
System	Dell Pro Max Tower T2 FCT2250	Single-workstation deployment; display remains attached.
CPU	Intel Core Ultra 9 285K; 24 cores / 24 threads; one NUMA node	Sidecar build and checkpoint I/O are CPU-heavy, but steady decode is GPU-bound.
Memory	64 GB installed; Linux sees 62.252 GiB; 2 x 32 GB DDR5-5600	Proven but tight. Qualification peaked around 52.3-52.6 GiB plus swap.
Swap	8.000 GiB file-backed swap	Needed operational headroom; a low-swap preflight once refused startup safely.
GPU	RTX PRO 6000 Blackwell Workstation Edition; SM120; 97,887 MiB VRAM	The remaining model and KV cache fit only because this is a 96 GB-class GPU.
PCIe	PCIe Gen5 x16 current and maximum	Important because every selected PLE row is read from pinned host memory through UVA.
Power	450 W configured cap	Long prefills reached roughly 452 W; decode qualification peaked lower.
Storage	WD_BLACK SN850X 2 TB; ext4	Checkpoint + sidecar + image consume about 178.34 GB decimal before logs and build headroom.

Pinned software

Layer	Exact version / identity
OS / kernel	Ubuntu 24.04.4 LTS / 6.17.0-1032-oem
NVIDIA	Driver 610.43.02; CUDA user-mode stack 13.x
Container	Docker 29.1.3; NVIDIA Container Toolkit 1.20.0
Image	lmsysorg/sglang@sha256:59f06adce6f91401adf443bd168d45fdb2044d77671fd591c7c57a29d851cbae
SGLang	0.0.0.dev1+gd91c3682b; commit d91c3682b0b429e4c70df63cd57f819588ce29b0
PyTorch / Triton	2.13.0+cu130 / 3.7.1
FlashInfer	0.6.17

Practical hardware recommendation

A nominal 64 GB host is proven, not comfortable. For a fresh build, 96 GB RAM is the sensible floor and 128 GB removes most swap pressure. Keep at least 200 GB free after the checkpoint download; 220 GB is safer if you retain receipts and temporary output.

MODEL COMPOSITION

The checkpoint already does half the job

The runnable artifact is not the raw BF16 source. It is RadixArk/Qwen3.8-Flash-Next-NVFP4, pinned at revision `7b719225242aacd3dbd3f9407468c2ee9a9d2594`. Its index totals 135,195,303,851 bytes of tensors (125.910 GiB). The model card describes roughly 180B total parameters, 48 decoder layers, 512 routed experts with top-10 routing, plus multimodal and hybrid recurrent/sparse-attention components.

What the checkpoint quantizes

Component	Checkpoint storage	Runtime placement
Routed MoE experts	ModelOpt NVFP4 W4A4, group 16	GPU
Attention, QSA, GDN	BF16	GPU
Shared experts, routers, embeddings, LM head	BF16	GPU
Vision and MTP tensors	BF16	GPU
PLE n-gram table	FP8 E4M3 + outer BF16 scalar	Pinned host RAM

GPU fit

SGLang reported a 81.29 GB GPU weight load and 9.91 GB available immediately afterward. The ready process later allocated a BF16 KV cache of 160,832 tokens (1.84 GB K + 1.84 GB V) and decode graphs. A representative healthy snapshot used 87,431 MiB, leaving 9,813 MiB free.

135.195 GB	81.29 GB	160,832	9,813 MiB
checkpoint tensors	logged GPU weights	allocated KV tokens	healthy free VRAM

Important distinction
The checkpoint is 135 GB on disk, but the complete PLE table is intentionally not resident in VRAM. Disk size, GPU residency, and host pinned-memory residency are three separate budgets.

THE ORIGINAL BLOCKER

Why stock PLE offload fails on 64 GB

SGLang's Qwen4Exp offload path keeps the entire PLE table in pinned CPU memory and gathers selected rows through UVA. That is the correct high-level architecture, but the stock table remains FP8 or BF16. On this host, the FP8 allocation failed before ordinary model weight loading, even after freeing roughly 51 GiB of host memory.

Property	Exact value
Partitions	128 synchronized checkpoint shards
Rows per partition	2,500,012
Total rows	320,001,536
Embedding width	160 values
Source dtype	FP8 E4M3, one byte per value
Pinned allocation	320,001,536 x 160 = 51,200,245,760 B = 47.684 GiB
Outer scale	BF16 scalar 0.00019931793212890625

Context tuning cannot solve this allocation
Reducing context, queue depth, KV dtype, or CUDA graph size changes GPU-side caches. It does not shrink the table allocated by the PLE constructor. The fix had to change the host representation itself.

Why generic CPU offload is not the answer

The Qwen4Exp server code explicitly treats PLE offload as a special pinned-memory path. Generic `--cpu-offload-gb` or layer-offload controls can stage the table back toward the device and conflict with the intended UVA gather. The working configuration uses only `--ple-offload-embedding` plus the custom packed representation.

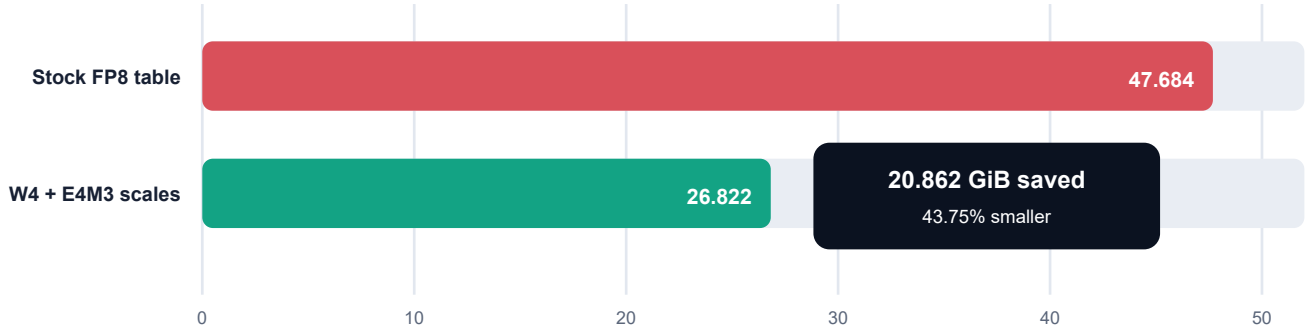
The 52 GB shorthand

Enthusiasts often call this a 52 GB table. The exact value is 51.200 decimal GB, which is 47.684 binary GiB. This report gives both units because mixing them can erase several gigabytes of apparent headroom.

W4 PLE SIDECAR

The memory breakthrough

Pinned PLE memory, GiB



Representation	Bytes per row	Total bytes	GiB
Source FP8	160	51,200,245,760	47.684
Signed INT4 values	80	25,600,122,880	23.842
E4M3 scales (10 x 1 byte)	10	3,200,015,360	2.980
Packed total	90	28,800,138,240	26.822

Exact savings
Every 16-value source block becomes 8 packed bytes plus one one-byte FP8 scale: 9 bytes instead of 16. The retained fraction is 56.25%; the table is exactly 43.75% smaller, saving 22,400,107,520 bytes (20.862 GiB).

Why group 16

Group 16 matches the natural block scale used by the NVFP4 ecosystem, produces ten scale bytes for a 160-wide row, and measured better than a single-scale row-wise INT4 scheme. A 98,304-row real-data sample measured NRMSE 0.08147 and cosine similarity 0.99631. That is promising, but it is still lossy quantization; behavioral and long-context gates remain mandatory.

EXACT ALGORITHM

Quantization and decode

For each group of 16 FP8 values, the builder chooses an asymmetric signed range through a shared symmetric-looking scale: positive values get seven codes and negative values get eight. The scale itself is rounded to E4M3 before integer quantization, so the builder and GPU kernel agree on the stored value.

```
# One 16-value block B, evaluated in float32
max_positive = max(clamp_min(B, 0)) / 7
max_negative = max(-clamp_max(B, 0)) / 8
scale = max(max_positive, max_negative, 2**-9)
scale_fp8 = cast_e4m3(scale)

q = round_to_nearest_even(B / float(scale_fp8))
q = clamp(q, -8, 7)

packed[j] = (q[2*j] & 0xF) | ((q[2*j+1] & 0xF) << 4)
decoded[i] = bf16(sign_extend(nibble[i]) * float(scale_fp8))
final_ple_row = decoded * 0.00019931793212890625
```

Nibble contract

Field	Contract
Integer range	Signed -8 through +7
Packing	Even dimension in low nibble; odd dimension in high nibble
Negative encoding	4-bit two's complement
Rounding	Torch round-to-nearest-even
Scale	One E4M3 byte per 16 dimensions
Output	BF16 selected rows, followed by the checkpoint's outer BF16 scale

What is bit-exact

The CPU builder, online CUDA packer, pinned runtime class, and Triton dequantization path were checked for byte/bit parity. That proves implementation parity. It does not mean the W4 table is identical to the source FP8 table.

BUILD ONCE, LOAD QUICKLY

Persistent sidecar and integrity contract

The production sidecar is a separate, immutable artifact. The pristine model directory remains untouched. The builder streams all 128 source partitions in 65,536-row chunks, calculates source and output hashes, writes the two binary tensors, and seals a canonical manifest.

```
python build_ple_w4_sidecar.py   --model-dir /path/to/Qwen3.8-Flash-Next-NVFP4   --output-dir /path/to/empty/
ple-w4-sidecar   --source-revision 7b719225242aacd3dbd3f9407468c2ee9a9d2594   --chunk-rows 65536   --threads 2
```

Artifact	Shape / size	SHA-256
qweight.u8	[320001536, 80] / 25,600,122,880 B	7e18b8dac400bda73...8150df3297
block_scales.f8	[320001536, 10] / 3,200,015,360 B	667736278723db00...35add7e09a1c
manifest.json	13,451 B	a11028a945bac40c...8731d7e1691

Runtime validation is fail-closed

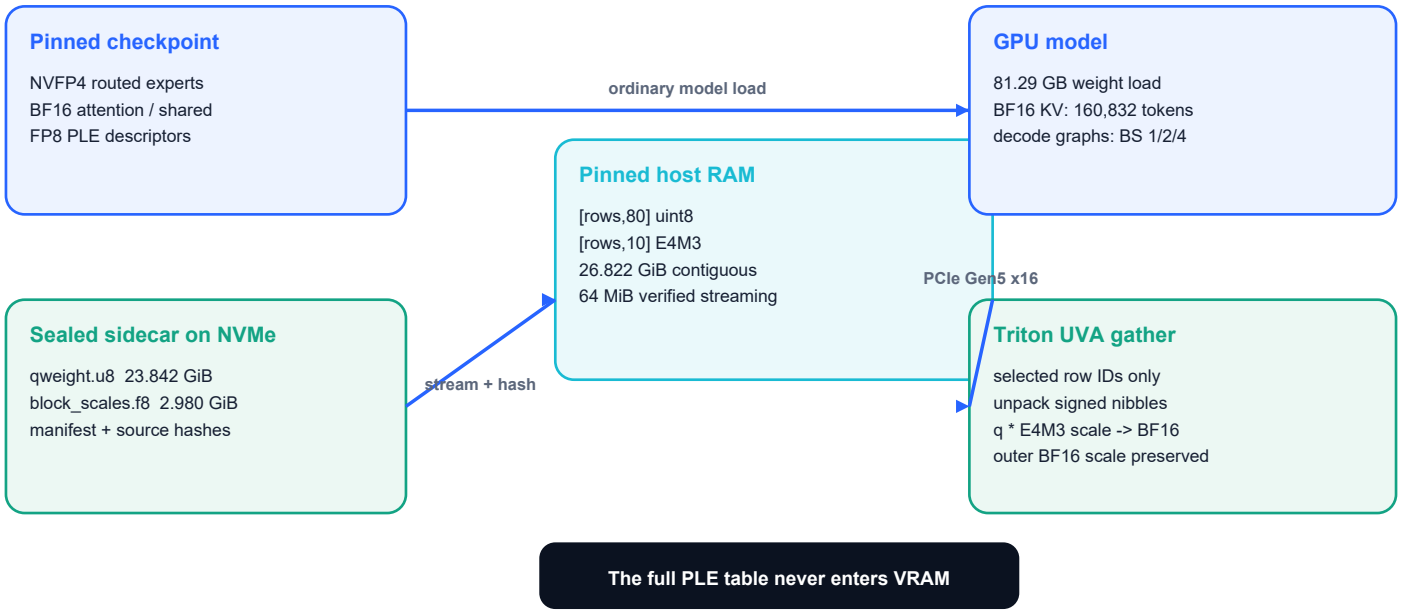
- Require the expected manifest digest and exact model revision, index, config, HF tree receipt, and raw-PLE digest through environment bindings.
- Reject symlinks, traversal, unsupported layout/version, wrong shape/dtype/size, duplicate shards, incomplete 128-shard coverage, and outer-scale mismatch.
- Stream each binary directly into its already-allocated pinned tensor in 64 MiB chunks while calculating SHA-256.
- Keep SGLANG_QWEN4_PLE_W4_SIDECAR_FALLBACK=0. A corrupt or stale artifact must close the port, not silently repack or serve a mismatched table.

Fine-tuning rule
Rebuild and reseal the sidecar for every modified checkpoint, even if a fine-tune was intended to leave PLE untouched. Never reuse a sidecar solely because its shape matches.

PINNED HOST UVA

Runtime data path

Runtime data path



Why ordinary safetensors mmap matters

SGLang still enumerates the 128 source PLE tensors so the loader can validate names, shapes, dtypes, shard indices, and coverage. With ordinary safetensors mmap, that enumeration is metadata-oriented and does not require copying the 51.2 GB source payload. Checkpoint prefetch, mmap-disable, or alternate fast-loading paths can page in the source and destroy the memory budget.

Why contiguous pinned tensors

The deployment uses two contiguous allocations: `uint8[320001536,80]` and `e4m3[320001536,10]`. This preserves a single stable UVA pointer for each tensor, keeps CUDA graph addresses stable, and avoids per-shard pointer tables and indirection.

PCIe behavior
Only requested rows are read over PCIe. Each 160-dimensional source row becomes 90 host bytes plus GPU-side dequantization work. The full PLE table remains pinned in host RAM for the life of the process.

BLACKWELL-SPECIFIC FIX

The SM120 attention patch

The pinned SGLang Qwen sparse-attention resolver admitted the TRT-LLM/FlashInfer decode path only on SM100. On the RTX PRO 6000 (SM120), it fell through to FlashAttention 4 / CuTe and hit a layout congruence failure. The local overlay broadens the gate and explicitly selects FlashInfer XQA on SM120.

```
from functools import lru_cache, partial
from sglang.srt.utils import is_sm100_supported, is_sm120_supported

if not (is_sm100_supported() or is_sm120_supported()):
    return None

from flashinfer.decode import trtllm_batch_decode_with_kv_cache

if is_sm120_supported():
    return partial(trtllm_batch_decode_with_kv_cache, backend="xqa")
return trtllm_batch_decode_with_kv_cache
```

Qualified kernel split

Subsystem	Qualified path	Reason
QSA decode	FlashInfer XQA on SM120	Avoids the incompatible FA4 fallback; XQA tests include compute capability 12.
QSA KV cache	BF16	FP8 KV failed on this arm; BF16 passed long-context and graphs.
GDN prefill	FlashInfer	Qualified at long context.
GDN decode / verify	Triton	Stable with this image and graph profile.
Mamba state	FP32	Conservative correctness choice after lower-precision conflicts.

Upstream status at capture
Qwen4Exp support PR #36497 and the SM120 fix were still open. Pin the exact image and overlays; do not assume a current pip package or main branch reproduces this behavior.

SGLANG CONFIGURATION

Exact production serving profile

```
sglang serve   --model-path /models/flash-next   --served-model-name qwen3.8-flash-next   --host 0.0.0.0 --
port 8002   --tp 1   --load-format safetensors   --quantization modelopt_fp4   --ple-offload-embedding   --
context-length 131072   --kv-cache-dtype bfloat16   --max-total-tokens 163072   --max-running-requests 4   --
max-queued-requests 16   --chunked-prefill-size 1024   --linear-attn-prefill-backend flashinfer   --linear-
attn-decode-backend triton   --mamba-ssm-dtype float32   --reasoning-parser auto   --tool-call-parser auto   -
-default-chat-template-kwargs '{"enable_thinking": false}'   --mem-fraction-static 0.94   --cuda-graph-config
    '{"decode":{"backend":"full","max_bs":4,"bs":[1,2,4]},      "prefill":{"backend":"disabled"}}'   --
disable-radix-cache
```

Deliberately off	Reason
TP > 1	Current sidecar supports TP1 and exactly one PLE module only.
FP8 KV	Failed on the qualified QSA arm; BF16 is the proven path.
Prefill CUDA graphs	Long/chunked prefills stay eager; decode-only graphs are stable.
Radix cache	Avoids hybrid QSA/GDN cache complexity and extra Mamba slot pressure.
Mixed / dynamic chunking	Mixed mode had QSA prefix-alignment risk; dynamic chunking did not apply at PP1.
Automatic truncation	Over-limit requests must fail clearly rather than silently losing prompt tokens.
Speculative decoding	Not part of this qualification; keep variables isolated.

DEPLOYMENT

Environment, mounts, and container contract

```
SGLANG_QWEN4_PLE_W4=1
SGLANG_QWEN4_PLE_W4_CHUNK_ROWS=262144
SGLANG_QWEN4_PLE_W4_SIDECAR=/sidecars/ple-w4
SGLANG_QWEN4_PLE_W4_SIDECAR_CHUNK_BYTES=67108864
SGLANG_QWEN4_PLE_W4_SIDECAR_FALLBACK=0
SGLANG_QWEN4_PLE_W4_SIDECAR_MANIFEST_SHA256=<sealed-manifest-sha>
SGLANG_QWEN4_PLE_W4_SOURCE_INDEX_SHA256=<pinned-index-sha>
SGLANG_QWEN4_PLE_W4_SOURCE_CONFIG_SHA256=<pinned-config-sha>
SGLANG_QWEN4_PLE_W4_SOURCE_TREE_SHA256=<pinned-tree-receipt-sha>
SGLANG_QWEN4_PLE_W4_SOURCE_PLE_SHA256=<raw-ple-sha>
SGLANG_QWEN4_PLE_W4_SOURCE_REVISION=<pinned-revision>
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True
```

Read-only bind mounts

- Pristine model directory -> /models/flash-next:ro
- Sealed W4 sidecar -> /sidecars/ple-w4:ro
- qwen4_exp.py packed-loader overlay -> exact SGLang source path:ro
- qwen4_ple_w4_sidecar.py strict loader -> exact SGLang source path:ro
- qwen_sparse_attn_backend.py SM120 XQA overlay -> exact SGLang source path:ro

Container settings

The production container uses all GPUs, host network, host IPC, local log rotation (50 MB x 3), no Docker restart, and no auto-remove. Model, sidecar, and overlays are mounted read-only. This captured launch does not set SGLang's available API-key option, so network access is controlled outside the process.

Do not expose port 8002 directly
This box binds 0.0.0.0 because access is controlled elsewhere on a private network. For a personal reproduction, prefer 127.0.0.1 or enforce a firewall/VPN/reverse proxy with authentication.

FAIL CLOSED BEFORE THE GPU

Startup guardrails

The managed service is intentionally conservative. It refuses to stop or replace another workload and checks every exclusion twice: once before hashing and again immediately before Docker receives the GPU.

Preflight	Production threshold / behavior
GPU exclusivity	Exactly one GPU; >=90,000 MiB free; no competing inference process/container
Host RAM	MemAvailable >=36 GiB before launch
Swap	SwapFree >=1 GiB before launch
Port	8002 must be free
Artifact identity	Image, overlays, model index/config/tree, manifest, qweight, and scales are SHA-pinned
Source contract	128 FP8 shards; exact rows, dtype, module prefix, outer scale, and revision
Recovery boundary	No automatic launch of another model; failed preflight exits 78
Readiness	Verify argv/env/mounts/log markers, /health, one model ID, then exact SIDECAR_OK generation

Systemd posture

The unit has a 420-second start timeout, 120-second stop timeout, and a two-attempt start limit per 30 minutes. Automatic restart remains disabled. A failed named container and incorrect OOM/SIGKILL success classification can make naive restart loops unsafe.

Not high availability
A bounded restart policy still needs forensic archive/name release, correct exit-status classification, repeated memory/GPU gates, and an attended fault drill. Restart=no is an explicit unqualified state, not an omission.

OPERATIONAL ENVELOPE

Memory and capacity at steady state

Budget	Observed value	Interpretation
Pinned PLE	26.822 GiB	Lifetime host allocation: qweight + E4M3 scales
Container RAM	~46.5 GiB steady	Includes pinned table and runtime allocations
Container RAM peak	~52.3-52.6 GiB	Qualification / production receipts; zero cgroup OOM
Container swap peak	~1.4-2.1 GiB	Host was shared and under pressure
GPU weight load	81.29 GB	SGLang load log
Ready GPU use	~87.4K MiB	Weights + caches + graphs; roughly 9.8K MiB free
BF16 KV	160,832 tokens	1.84 GB K + 1.84 GB V on current boot
Mamba state	4 slots	Four concurrent requests share the state budget

Context math

The command requests 163,072 shared tokens; the runtime profiled approximately 160,832 on the current boot. The API context ceiling is 131,072 per request. Four request slots therefore do not mean four simultaneous 131K prompts. Four approximately 32K prompts fit and passed together; four maximum-length prompts do not fit the shared pool.

131,072	160,832	4 x ~32K	HTTP 400
per-request ceiling	current shared KV pool	qualified concurrent prompts	over-limit behavior

Capacity is boot-sensitive

The profiled KV pool varied from roughly 155K to 161K as startup headroom changed. Treat max-total-tokens as a request to the allocator, then record the resolved token count from the log.

PREBUILT SIDECAR PAYOFF

Cold-start profile

Qualified cold-start timeline

Online packing was ~354 s; sidecar startup is ~198-204 s

Stage	Observed
Sidecar build (one time)	550.112 s
Verified sidecar load	15.4-16.4 s
All 128 source descriptors verified	~124 s after container start
Weight load complete	108.6-113.5 s stage; 81.29 GB GPU memory
Decode graph capture	~1.8 s
Container start -> ready	~198-204 s
Prior online-pack start -> ready	~354 s

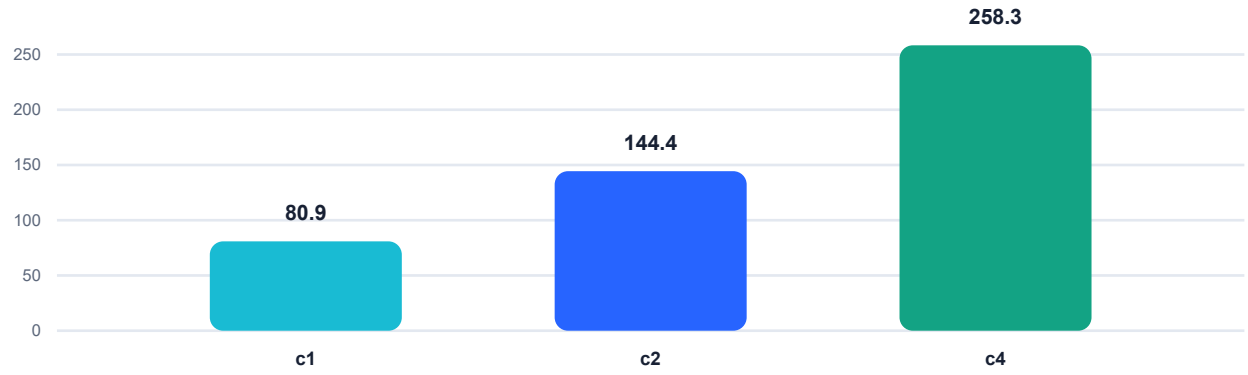
Where the speedup comes from

The PLE phase fell from about 309.5 seconds of online quantization to about 16 seconds of verified binary loading - roughly 19.2x for that stage. Full startup improved by about 44% because the remaining 135 GB checkpoint still has to be mapped and loaded.

COMPARABLE 131K PROFILE

Decode performance

Sealed 131K-profile greedy throughput (aggregate tok/s)



512-token decode; two-repeat medians; decode CUDA graphs enabled

Concurrency	Aggregate tok/s	TTFT p50	TTFT p95	E2E p50
1	80.869	92 ms	94 ms	6.330 s
2	144.389	139 ms	191 ms	7.092 s
4	258.320	127 ms	167 ms	7.927 s

The best c4 repeat was 260.764 tok/s. The preceding identical 131K online-packed arm measured 262.219 tok/s instruct and 265.008 tok/s thinking, placing the prebuilt sidecar within roughly 1.5%. That is expected: the sidecar changes startup loading, not the steady-state gather kernel.

Do not compare against the 320 tok/s headline
The 319.825 tok/s c4 instruct result came from an 8,192-token context and 8,192-token shared pool. It is a useful ceiling, not an apples-to-apples production result. The 131K allocation carries materially different memory pressure.

Power and thermal observations

The quick throughput gate peaked around 380.7 W and 66 C. Long retrieval telemetry reached about 452.5 W at a 450 W cap and 61 C. No thermal or hardware throttle was recorded in the sealed qualification.

VALIDATION

Long context, correctness, and honesty

Gate	Result
119K retrieval	119,391 server tokens; exact 3/3 codes; 17.272 s
130K boundary	130,391 server tokens; exact 3/3 codes; 18.151 s
130,502-token prompt	Exact expected answer; HTTP 200
131,094-token prompt	Correctly rejected with HTTP 400
4 x ~32K retrieval	4/4 exact; 16.802 s wall; median TTFT 11.323 s
119.5K + 512 decode	TTFT 13.959 s; 81.323 decode tok/s; 20.242 s total
4 x ~32K + 512	4/4; 73.57 aggregate tok/s
Behavior gate	13/13 exact response, math, logic, JSON, tools, stop, reasoning separation, retrieval, burst
Production APIs	Exact Chat and Responses markers with nonzero output tokens

Known behavioral caveat

A broader 16-case sample initially scored 15/16 because the model emitted one of two requested parallel weather calls. A corrected prompt later produced both calls with unique IDs and completed the continuation correctly. This is evidence of prompt-sensitive parallel-tool behavior, not a claim of perfect tool reliability.

Known scheduler caveat

When a short request arrived 0.5 seconds after a 119.5K prefill, the short request waited 13.502 seconds for its first token. Output remained correct, but head-of-line fairness failed. Chunked prefill size 1,024 is retained because current scheduling gives the active long prefill priority; larger chunks cannot provide true preemption.

Quality scope
These are engineering acceptance gates, not a full academic evaluation. Before publishing a derivative checkpoint, add perplexity and task suites appropriate to the intended use, plus multimodal tests if vision matters.

FOR OTHER ENTHUSIASTS

Reproduction runbook

1. Prove the hardware floor

Use a free 96 GB-class Blackwell GPU. Confirm SM120, PCIe link width, driver/toolkit compatibility, at least 64 GB host RAM plus swap, and at least 200 GB usable storage. 96-128 GB host RAM is strongly preferred.

2. Pin and download the checkpoint

Use hf download with the exact revision into a new directory. Verify model index/config and repository identity. The tested card calls the artifact a private candidate although the repository is currently public and ungated; comply with the Qwen license before use or redistribution.

3. Pin the container by digest

Pull the exact 13.95 GB SGLang image digest. Record contained SGLang, PyTorch, Triton, and FlashInfer versions. Do not substitute a mutable nightly tag.

4. Establish the stock failure

Run the exact Qwen4Exp PR-head path with --ple-offload-embedding on an isolated GPU. On a 64 GB host, expect the full pinned FP8 PLE allocation to fail. Do not tune context to hide a constructor-time host OOM.

5. Build and test the downstream code

Apply the packed PLE class, Triton gather, strict sidecar loader, loader-hook verification, and SM120 XQA overlay. Run synthetic corruption tests, real-row CPU/CUDA byte parity, and the actual pinned-class gather test.

6. Build the sidecar once

Quantize the pristine FP8 table in chunks, record per-shard and combined raw hashes, seal qweight/scales/manifest, then independently verify every output digest.

7. Launch fail-closed

Use TP1, ordinary safetensors mmap, read-only mounts, fallback=0, BF16 KV, FP32 Mamba state, XQA QSA decode, and decode graphs BS1/2/4. Keep competing GPU jobs off.

8. Qualify before sharing

Gate exact output, JSON/tools, reasoning separation, 119K and 130K retrieval, over-limit rejection, c1/c2/c4 throughput, GPU/host telemetry, and cold restart. Snapshot every hash and resolved runtime setting.

Versioned reproduction package

The public GitHub release supplies the exact downstream builder, sidecar loader, qwen4_exp patch chain, SM120 patch, tests, and sanitized receipts at sealed hashes. The companion Hugging Face repository supplies the 28.8 GB table. Verify both checksum files; this remains an experimental pinned recipe rather than a stock SGLang command.

LESSONS LEARNED

Dead ends and portability limits

Attempt / assumption	Observed outcome	Decision
Stock FP8 pinned PLE on 64 GB	OOM before normal weight load	Change host representation
BF16 PLE candidate	Would be about 102.4 GB for the same table	Reject for this host
Context reduction	Does not change constructor-time PLE allocation	Not a fit fix
Generic CPU offload	Conflicts with PLE's dedicated pinned-UVA path	Do not combine
FP8 KV	QSA dtype/path failure on this arm	Use BF16 KV
SM120 FA4 fallback	CuTe layout congruence failure	Force XQA
Tensor parallel > 1	Not implemented for the sealed sidecar	TP1 only
Radix / mixed chunk	Hybrid state and prefix-alignment risk	Disable
Naive automatic restart	Failed container name and exit classification can block or loop	Keep Restart=no until fault-tested

Portability

- SM120 is the only GPU architecture tested. XQA availability and page/head geometry must be revalidated on another GPU family.
- 96 GB VRAM is the proven class. An 80 GB card is unlikely to fit the logged 81.29 GB weight load plus KV and graphs without additional changes.
- A PCIe Gen5 x16 link is part of the observed performance envelope. Slower links may expose the host-gather cost.
- The model advertises 262K context, but this deployment qualifies 131K for headroom and concurrency.
- The service defaults thinking off for shared clients; explicit thinking requests were tested separately.

OPERATIONAL HANDOFF

Release, fine-tuning, and verification

The public implementation, patches, tests, receipts, and current companion-artifact location are preserved at:

```
github.com/lEWFkRAD/qwen38-rtx-pro-6000/tree/Cloud1/
flash-next-w4-ple
```

Identity	SHA-256 / value
Snapshot checksum list	214ed12f3de5442ae9346b680f49354541eea62973c5df038bc58fb80b391f34
Model revision	7b719225242aacd3dbd3f9407468c2ee9a9d2594
Model index	da5ca9c3b65e48e151329e64e141c2fa700bf2f99aec53cc014e4b52a6ff7a84
Config	e765305daba0951974308f4d32c075b52a6a45974730d273f2216718a994d624
Raw FP8 PLE	b070f9644adf93794d8a1030584ab705809387e64396a9327a68fa3a3a6666b3
Sidecar manifest	a11028a945bac40c7a2d5f41f21c829a08f5d531c39184dda2a7c8731d7e1691
Production qwen4_exp overlay	b54de0a07a16a7a3070aabead3c53b80f108d89528c30763ca8e032f330d97ac
Sidecar loader module	9dbace396f69ca2a319c7ae9cf74380549b62b6cdcc9f88135776552c245bb68
QSA overlay	584e2acdce11c6e1e6dc50b9f61ca8018a3634e1bafb3d079b367fc30d1f7634

```
git clone --branch Cloud1   https://github.com/lEWFkRAD/qwen38-rtx-pro-6000.git
cd qwen38-rtx-pro-6000/flash-next-w4-ple
sha256sum -c SHA256SUMS

# Obtain HF_REPO_ID from this release README, then:
hf download "$HF_REPO_ID" --local-dir /path/to/ple-w4-sidecar
cd /path/to/ple-w4-sidecar
sha256sum -c SHA256SUMS
sha256sum -c manifest.sha256
```

Fine-tuning workflow

Clone the pristine checkpoint into a new content-addressed snapshot, fine-tune only inside that snapshot, regenerate the W4 PLE sidecar, rerun parity and quality gates, then assign a new manifest and runtime overlay. Never overwrite the qualified base or its sidecar.

REFERENCES

Public sources and provenance

#	Primary source
1	Public implementation and field report https://github.com/lEWFkRAD/qwen38-rtx-pro-6000/tree/Cloud1/flash-next-w4-ple
2	RadixArk checkpoint card https://huggingface.co/RadixArk/Qwen3.8-Flash-Next-NVFP4
3	Pinned RadixArk revision https://huggingface.co/RadixArk/Qwen3.8-Flash-Next-NVFP4/tree/7b719225242aacd3dbd3f9407468c2ee9a9d2594
4	Pinned configuration https://huggingface.co/RadixArk/Qwen3.8-Flash-Next-NVFP4/blob/7b719225242aacd3dbd3f9407468c2ee9a9d2594/config.json
5	Qwen3.8-Flash-Next source repository https://github.com/QwenLM/Qwen3.8-Flash-Next
6	Qwen architecture README https://github.com/QwenLM/Qwen3.8-Flash-Next/blob/main/README.md
7	SGLang Qwen4Exp support PR #36497 https://github.com/sgl-project/sglang/pull/36497
8	Exact PR-head Qwen4Exp source https://github.com/sgl-project/sglang/blob/73a255206f916366c8d26d4022f82ddfb0ab558d/python/sglang/srt/models/qwen4_exp.py
9	SGLang SM120 QSA issue #36531 https://github.com/sgl-project/sglang/issues/36531
10	SGLang proposed SM120 fix #36556 https://github.com/sgl-project/sglang/pull/36556
11	SGLang FP8 KV issue #36545 https://github.com/sgl-project/sglang/issues/36545
12	FlashInfer v0.6.17 https://github.com/flashinfer-ai/flashinfer/releases/tag/v0.6.17
13	FlashInfer XQA source https://github.com/flashinfer-ai/flashinfer/blob/v0.6.17/flashinfer/xqa.py
14	FlashInfer XQA tests https://github.com/flashinfer-ai/flashinfer/blob/v0.6.17/tests/attention/test_xqa_batch_decode.py
15	NVIDIA RTX PRO 6000 family https://www.nvidia.com/en-us/products/workstations/professional-desktop-gpus/rtx-pro-6000-family/
16	NVIDIA Container Toolkit install guide https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/install-guide.html
17	Hugging Face CLI pinned downloads https://huggingface.co/docs/huggingface_hub/main/en/package_reference/cli
18	SGLang installation and Docker guidance https://github.com/sgl-project/sglang/blob/main/docs/docs/get-started/install.mdx

Published evidence

Exact measurements, hashes, timelines, and behavior results originate in the sealed Onyx qualification snapshot. The public GitHub release now contains the downstream implementation, reproducible patch chain, manifest contract, and sanitized benchmark receipts; the companion binary table is bound by the manifest SHA-256 above.

Prepared as a reproducibility-oriented engineering record. Hardware serials, UUIDs, credentials, and private network routing are omitted. All byte counts, revisions, and benchmark qualifications are preserved.